I. Introduction

abc.

II. Implementation Process

A. Forwarding and packet headers

After constructing the network topology described above, we initially attempted to generate and transmit an Internet Control Message Protocol (ICMP) ping packet embedded with a source routing header. Given that modifying packet headers exclusively via command-line utilities presents significant challenges within the Mininet environment, we utilized a Scapy script executing on a host node to facilitate this procedure.

Within the Scapy framework, a custom packet with nested Ether-Type, 802.1Q, IPv4, and ICMP headers can be instantiated via the command

```
pkt = Ether() / Dot1Q() / IP() / ICMP(),
```

and subsequently transmitted using `sendp(pkt, iface="h1-eth1")`, where `iface` designates the interface connecting the host to the switch. Protocol-specific parameters can be passed as arguments during function invocation to configure the headers. Specifically, the egress port must be assigned to the `vlan` parameter within `Dot1Q()`, which constitutes the core mechanism for implementing source routing via OpenFlow. For multi-hop paths involving multiple switches, successive `Dot1Q()` layers must be encapsulated, with the `vlan` parameter of each layer mapped to the egress port of the respective switch. The corresponding syntax is defined as follows:

```
pkt = Ether() / Dot1Q(vlan = out_port_sw1)

    / ... / Dot1Q(vlan = out_port_swn)
```

Additionally, the source and MAC addresses must be supplied to `Ether()`, while the corresponding source and destination IP addresses are passed to `IP()`. Crucially, since the forwarding entries deployed on the switches do not incorporate specific MAC or IP criteria, the actual forwarding trajectory of the packet remains entirely independent of the addressing information contained within its network headers.

Furthermore, the synthesized packet header structure can be showed by invoking the `pkt.show()` method.

Next, we formulate the forwarding rules and deploy them to the switches within `controller.py`. In contrast to the Reactive Routing approach implemented in Tutorial 2, the proposed architecture mandates that all forwarding rules be pre-configured at network initialization. Consequently, the proactive flow provisioning mechanism leverages the `ofp_event.EventOFPSwitchFeatures` event as the trigger, rather than relying on the "packet-in" event. For the match fields, we specify the VLAN ID corresponding to a given port. Concurrently, two consecutive actions are defined: "PopVlan", which removes the outermost 802.1Q header and its encapsulated VLAN ID information, and "Output", which directs the packet to the physical port associated with that distinct VLAN ID. Utilizing the predefined maximum port capacity of each switch, these flow entries can be systematically populated in bulk.

After that, the implementation described above was tested. Upon initializing the network topology, the Scapy script was executed on host `h1` to initiate ICMP ping requests to host `h2` via diverse routing paths. Inspection of the flow table on switch `s1` revealed three successfully installed flow entries; their corresponding match criteria and actions conformed precisely to the predefined specifications. Furthermore, the active packet counters indicated that real-time packet processing had occurred, thereby verifying the successful deployment of the flow tables.

Subsequently, Wireshark was deployed to capture and analyze the ICMP traffic at multiple observation points along the forwarding path. The observations demonstrated two critical findings:

- The synthetic packets were successfully delivered to the target destination, `h2`.
- The outermost 802.1Q header was progressively removed as the packet traversed each switch in the path.

Consequently, the packet arrived at `h2` without any 802.1Q encapsulation, rendering its structural format identical to a conventional ICMP packet generated by a standard native `ping` command in Mininet.

If `h2` responds to this ICMP echo request, the resulting reply packet cannot be successfully forwarded by the switches due to the lack of a valid source routing header. To circumvent this limitation, a similar Scapy script is implemented and deployed on host `h2`. Operating symmetrically to the script on `h1`, this script continuously sniffs inbound ICMP traffic. Upon detecting a valid packet, it dynamically generates and transmits a corresponding reply packet encapsulated with a source routing header that includes the return path, whose information is inherent to `h2` itself.

III. Discussion

Switches employing conventional forwarding method typically maintain massive number of flow entries. While the source routing approach proposed by Huang et al. significantly decreases these entries, their forwarding plane includes the flow entries for normal forwarding, the initialization of source routing headers, and of the fast-failover groups. In contrast, our proposed methodology requires switches to retain exclusively normal forwarding entries, specifically restricted to "pop" and "output" actions, since that the host end-node inherently possesses the complete path topology before packet transmission.

Consider an n*n matrix network, the total flow entries of the work of Huang et al. are represented as:

$$F_{SR-SDN} = F_{\text{working\_path}} + F_{\text{backup\_path}},$$

$$F_{\text{working\_path}} = n^2 \times p,$$

$$F_{\text{backup\_path}} = \sum_{i=2}^{n} \sum_{j=1}^{i} [(i + j - 1) \times M_{n,i,j}],$$

where

$$M_{n,i,j} = (n - i + 1) \times (n - j + 1) \times \sigma_{i,j};$$

$$\sigma_{i,j} = \begin{cases} 8, & \text{if } j \neq 1, i \neq j \\ 4, & \text{if } i = 1 \text{ or } i = j \end{cases}.$$

Where $F_{working\_path}$ are the flow entries for normal forwarding and $F_{transient\_plane}$ represents the flow entries for initialing source routing header, and for finding and switching to the backup path. The parameter p is the physical port number of a switch, it should be set to $p = 4$ in the considered grid topology.

In our work, the total number of flow entries does not include those for transient plane:

$$F_{our} = F_{\text{working\_path}} = n^2 \times p,$$

where p is the physical port number of a switch. Since each need 1 flow entry for each port, this form represents the entries' number for n*n switches.

Consequently, in comparison with the framework, our approach has the minimum number of flow entries for switches within an identical network topology, thereby achieving superior reliability and scalability.

References

[1] –